One network for renders and video: vendor-neutral neural temporal super-resolution and low-latency frame generation trained on commodity cloud GPUs

Parth Mital

*School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, 632014, Tamil Nadu, India.

Corresponding author(s). E-mail(s): parth.mital2023@vitstudent.ac.in;

Abstract

Neural upscaling and frame generation now set image quality in many games, but the strongest commercial systems need one vendor's recent GPUs, and research systems need engine data that ordinary video lacks. We present NeuralSS, an open system that serves both cases with one model per task. Its temporal super-resolution network is a recurrent, re-parameterisable convolutional network with a built-in motion estimator that runs only when the engine supplies no motion vectors. Its frame-generation network fine-tunes the open RIFE 4.26 interpolation network, adds engine hints through zero-initialised channels, extrapolates with a late-latched camera prior and copies user-interface pixels from the newest real frame. Both are trained in about ten hours each on two NVIDIA T4 GPUs from public datasets, after copy detection that left no duplicate across splits, and both export to ONNX. On five test suites the 3x super-resolution student (16.1 GMAC per 1080p frame) beats bicubic upscaling on ordinary video by 1.2 to 2.2 dB PSNR-Y and an analytic temporal anti-aliasing upscaler by 2.8 to 4.9 dB with engine data, with lower temporal error. The largest frame-generation tier beats unchanged RIFE 4.26 on the Middlebury interpolation set (34.23 against 33.34 dB) and frame repetition in extrapolation by 1.9 to 5.6 dB, but the 12.7-GMAC tier is still 1.8 dB below RIFE, the cut detector false-alarms on multi-frame gaps, and eager PyTorch on a T4 needs 27.7 and 48.1 ms per 1080p frame, above real-time budgets. We report these limits with the gains.

Keywords: Temporal super-resolution, Frame generation, Video frame interpolation, Real-time rendering, Latency, Efficient neural networks

1 Introduction

Games increasingly render fewer pixels than they display. A temporal upscaler reconstructs each output frame from a jittered low-resolution render and the history of earlier frames, and a frame generator inserts synthetic frames between rendered ones. Neural versions of both now ship in vendor SDKs, but the strongest paths need recent hardware of one vendor: DLSS 4 multi-frame generation relies on RTX 50 Series GPUs[1], and AMD's machine-learning upscaling and frame generation require RDNA 4 GPUs, with analytical fallbacks

[1] https://research.nvidia.com/labs/adlr/DLSS4, accessed 8 November 2026.

on older ones[2]. Intel's XeSS super-resolution runs on any GPU with Shader Model 6.4[3]. Research systems are open but assume a full set of engine buffers [1–3], so they cannot serve captured or streamed video, where only colour exists. Video frame interpolation networks such as RIFE [4] work from colour alone but ignore the motion vectors, depth and user-interface (UI) layers that an engine can provide, and they do not extrapolate.

This paper asks whether one compact model per task can serve both inputs, engine renders with motion vectors and plain video without them, at a cost that fits integrated and entry-level GPUs, when trained with free cloud compute and public data only. The real-time constraints shape every design choice. Each network has tiers sized in multiply-accumulates (MAC) per 1080p output frame. Frame generation is designed around input latency rather than frame rate, so it supports extrapolation, which holds no rendered frame back. Both networks reduce to convolutions, pixel shuffles and bilinear warps that export to ONNX for vendor-neutral runtimes.

The contributions are:

1. A recurrent temporal super-resolution (TSR) network with a built-in motion estimator, trained on mixed render-like and plain-video degradations so that one model serves both, with tiers from 6.2 to 156 GMAC per 1080p frame at 3x (Section 4).
2. A frame-generation (FG) network that fine-tunes RIFE 4.26 and adds engine hints through zero-initialised channels, so that a fresh model reproduces RIFE exactly, plus a UI mask, a late-latched camera prior for extrapolation, and a scene-cut gate (Section 5).
3. A leakage-audited data pipeline over 12,300 public items and an evaluation protocol that tests every suite in an engine mode and a plain-video mode, with temporal, UI, cut and latency measures (Sections 3 and 6).
4. Measurements on two Kaggle T4 GPUs that include the failures: the small FG tiers stay below RIFE, the plain-video cut gate false-alarms, and eager latency exceeds the tier budgets (Sections 7 and 8).

[2] https://gpuopen.com/learn/amd-fsr-redstone-developers-neural-rendering/, accessed 8 November 2026.

[3] https://github.com/intel/xess, accessed 8 November 2026.

Figure 1 summarises both networks.

Code, notebooks, metrics and weights are public (Section 9).

2 Related work

Temporal anti-aliasing and upscaling. Temporal anti-aliasing reprojects an accumulated history with motion vectors and rectifies it against the current frame's neighbourhood to suppress ghosting, and temporal upsampling extends the same idea to lower render resolutions [5]. Our analytic baseline follows this recipe.

Neural supersampling for rendering. NSRR reconstructs high-resolution frames from the colour, depth and dense motion vectors of the current and previous frames [1]. Later work combines denoising and supersampling in a temporally stable network [6], reduces cost on a new gaming dataset [7], fuses high-resolution G-buffers with low-resolution colour [2], demodulates lighting from materials [8] and decouples G-buffer guidance [9]. All assume engine buffers at inference. Our TSR network uses them when present and estimates motion itself when they are absent.

Frame generation for rendering. Interpolation for rendered content can use auxiliary buffers [10], but it holds a rendered frame back. Extrapolation avoids that latency: ExtraNet uses G-buffers of the new frame and in-paints disocclusions [3], ExtraSS joins extrapolation with supersampling [11], space-time supersampling treats both in one network [12], GFFE removes the need for the new frame's G-buffers [13], and Mob-FGSR targets mobile GPUs [14].

Video frame interpolation. RIFE estimates intermediate flows coarse to fine with an IFNet at an arbitrary time step and runs in real time [4]. IFRNet [15], FILM [16] and EMA-VFI [17] trade more computation for accuracy or large motion. Logos, UI and subtitles break interpolation, and a discontinuity map with figure-text mixing copies such regions from the previous frame instead of interpolating them [18]. We build on RIFE 4.26 because its weights are open and it is cheap enough to tier.

Efficient networks and temporal metrics. Structural re-parameterisation trains multi-branch blocks and deploys one 3×3 convolution [19, 20], and sub-pixel convolution upsamples cheaply [21]. Recurrent video super-resolution

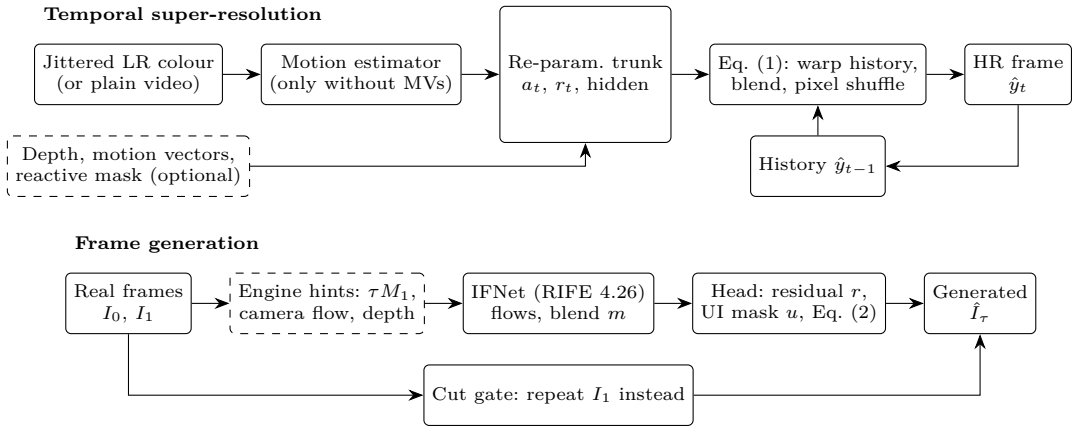


Fig. 1 Overview of the two networks. Dashed boxes are engine inputs that are used when present and dropped at random in training. Both networks reduce to convolutions, pixel shuffles and bilinear warps for ONNX export

propagates features across frames [22]. TecoGAN defines the tOF and tLP temporal metrics we report [23].

3 Data and leakage control

Training and test data come only from public sources: DIV2K [24] and other high-resolution stills, 1080p game frames, REDS [25], Vimeo-90K septuplets and triplets [26], MPI Sintel with exact flow and occlusion [27], TartanAir with exact flow and depth from Unreal Engine environments [28], CARLA [29] renders from GameIR with native 720p and 1440p frames [30], the Vid4 sequences (calendar, city, foliage and walk) evaluated by Liu and Sun [31], and 1080p Vimeo videos. Low-resolution inputs are generated from high-resolution frames, so every sample has ground truth.

Copy detection runs in two stages, because global similarity alone cannot tell the same picture from a similar one. Candidates are the 16 nearest neighbours of each keyframe by self-supervised DINOv2-small embeddings outside its leakage group plus perceptual-hash pairs, and a candidate becomes a duplicate edge when SIFT matches [32] with the ratio test and one RANSAC affine fit [33] cover at least 10 cells of an 8×8 grid, or when its embedding or hash similarity passes a stricter threshold. Thresholds are calibrated on 1,500 synthetic copies against 2,930 random and 998 hard negatives (recall 0.967 at precision 0.999). Leakage groups (a video, a Sintel scene family, a TartanAir environment, a CARLA town) are the unit of the split, and any training item linked to a test item by a duplicate edge is dropped. An audit then compares every validation and test keyframe with its nearest training keyframes. On 12,300 items the split is 8,874 training, 1,087 validation and 1,432 test items, with 756 dropped and 151 filtered for quality. The first audit round found 24 duplicates that the search missed and the second found none.

A GPU renderer adds synthetic game-like sequences with exact ground truth. A background and a foreground layer with procedural alpha (blobs, wires one or two texels wide, fences) move on their own affine paths at four speed classes (static, slow, medium, fast) with optional reversal. Particles write no motion vectors and set a reactive mask, as in real engines. Low-resolution frames take one jittered sample per pixel at Halton (2, 3) offsets without mip filtering, so thin geometry aliases, and targets use four samples per pixel. For FG the renderer adds HUD tiles, subtitle changes, menus and scene cuts.

4 Temporal super-resolution

Inputs. At render resolution $h \times w$ the network receives the jittered colour, depth and normals derived from it, motion vectors to the previous frame, a depth-based disocclusion cue, the reactive mask, exposure and jitter offsets, flags for depth availability and motion quality (1 exact, 0.5 estimated, 0 none), an input-kind flag (render or video), the previous output warped by the motion vectors and folded to render resolution by

space-to-depth, and a warped recurrent hidden state.

Motion without engine data. Every tier contains a three-level IFNet motion estimator [4] that computes motion vectors from the previous and current colour frames whenever a sample has none. Its output enters the trunk with motion quality 0.25. It is supervised by exact or RAFT [34] motion, and during the first quarter of training samples without motion vectors warp the history with the label motion at a falling probability (teacher forcing), so that the trunk does not learn to distrust history while the estimator is poor. With engine motion vectors the estimator is skipped.

Trunk and output. The trunk is a chain of re-parameterisable blocks (a 3×3, a 1×1, an identity and fixed Sobel and Laplacian filters with learned projections) folded into one 3×3 kernel [20], with an optional half-resolution U-Net level. The side branches start at zero, which kept activations bounded in FP16. The network predicts a per-pixel history weight a_t and a residual r_t at render resolution, unfolds them by pixel shuffle [21], and forms

$$\hat{y}_t = a_t \odot \mathcal{W}(\hat{y}_{t-1}, \mathbf{m}_t) + (1 - a_t) \odot \mathcal{U}(x_t) + r_t, \quad (1)$$

where $\mathcal{W}$ warps by the upsampled motion $\mathbf{m}_t$ and $\mathcal{U}$ is a jitter-aware bilinear upsampling of the colour x_t. A zero-initialised two-layer refinement at output resolution follows in all tiers but the smallest. Equation (1) is a learned form of temporal accumulation with rectification: a_t should approach 0 at disocclusions and particles and 1 on static content. After a reset $a_t = 0$.

Training. Each batch mixes about 60 percent synthetic and 40 percent real windows. Thirty percent of synthetic and 60 percent of real samples become plain video: anti-aliased bicubic (cubic convolution [35]), bilinear or area downscaling, blur, noise and 8-bit quantisation, without jitter, depth or motion vectors. Render samples lose their motion vectors with probability 0.25 and depth with 0.3. Six-frame unrolls on 192×192 output patches are the default, and every fourth step is a 16-frame synthetic unroll at 96×96 so that static scenes keep converging. The per-frame loss is a Charbonnier penalty plus 0.1 times a gradient loss, 0.05 times an L1 loss on Fourier magnitudes, 0.05 times LPIPS [36] on the last frame, and 0.5 times a temporal-change loss $\|(\hat{y}_t - \mathcal{W}\hat{y}_{t-1}) - (y_t - \mathcal{W}y_{t-1})\|_1$ over pixels visible in both frames, plus 0.1 times the motion loss. Training uses AdamW [37], FP16 autocast with loss scaling [38], a cosine schedule over a wall-clock budget, and an exponential moving average of the weights. Distillation from the teacher gave no measurable gain in two earlier runs and is off.

Table 1 Network tiers: fused parameters, multiply-accumulates per 1080p output frame (GMAC, warps excluded) and measured T4 latency (fused FP16, eager PyTorch, cuDNN autotuning). TSR figures are for 3x and include the motion estimator, which runs only without engine motion vectors (latency with engine vectors in brackets). Tiers without latency were not trained

Task	Tier	Params	GMAC	T4 ms
TSR	`igpu`	138K	6.2	16.3 (10.0)
	`low`	328K	16.1	27.7 (21.0)
	`mid`	653K	28.1	–
	`high`	1.41M	69.0	–
	teacher	3.40M	156.1	58.7 (50.1)
FG	`igpu`	0.45M	9.4	46.8
	`low`	0.95M	12.7	48.1
	`mid`	2.29M	44.3	106.6
	`high`	5.71M	76.7	120.4

Table 1 lists the tiers. The teacher is the same architecture at 64 channels and serves as the quality ceiling.

5 Frame generation

Motion from colour. Given real frames I_0 and I_1 and a target time τ ($0 < \tau < 1$ interpolates, $\tau = 1.5$ extrapolates), an IFNet with RIFE 4.26's design estimates the flows $F_{\tau\to0}$ and $F_{\tau\to1}$, a blend logit and features over five blocks from 1/16 to full resolution. The `high` tier is RIFE 4.26's network and loads its weights, smaller tiers are narrower and inherit leading channel slices. Each tier estimates flow on at most 576 (`igpu`, `low`) or 1152 (`mid`, `high`) image lines, halving taller frames, so training crops always use full-resolution flow.

Engine hints. When an engine supplies the motion vector M_1 of I_1 (displacement to I_0), linear-motion priors τM_1 and $(\tau-1)M_1$, a camera-only flow sampled from the latest camera input, the depth of I_1 and availability flags enter every flow block as extra channels whose weights start

at zero. A fresh `high` model therefore reproduces RIFE 4.26 (maximum difference 3×10^{-6} against RIFE's own code), and fine-tuning learns how far to trust each hint. Training drops the hints at random so that one model serves engines and video.

Output and UI. A half-resolution head on the warped frames predicts a residual r and a UI logit u, and

$$\hat{I}_\tau = u \odot I_1 + (1-u) \odot \big(m \odot \mathcal{W}(I_0, F_{\tau\to 0}) + (1-m) \odot \mathcal{W}(I_1, F_{\tau\to 1}) + r\big), \qquad (2)$$

where m is the blend weight. Pixels classified as UI are copied from the newest real frame instead of being interpolated. If the engine provides HUD-less frames and a UI layer, the layer is composited on every displayed frame instead. The feature encoder depends on one frame only, so a running pipeline computes it once per real frame.

Cuts. A cut gate repeats the newest real frame instead of generating. With motion vectors its score is the luma-histogram distance plus a motion-compensated difference, without them the histogram distance alone. Each score has a threshold calibrated for 1 percent false alarms on continuous synthetic pairs.

Training. Synthetic samples provide exact intermediate flows, disocclusions, HUD tiles with subtitle changes, menus, cuts and camera reversals. Real samples are four-frame quads with RAFT flows, half of them without motion vectors, and a quarter of real interpolation samples are 3x multi-frame generation (targets at $\tau = 1/3$ and $2/3$). The loss is a Charbonnier penalty plus 0.5 times a census loss [39], 0.1 times LPIPS, 0.01 times a flow L1 loss, 0.2 times a binary cross-entropy on the UI mask and 0.5 times distillation. The `high` tier is fine-tuned from RIFE 4.26 first (learning rate 10^{-4}) and then distils its flows and outputs into the smaller tiers (4×10^{-4}).

6 Experimental setup

Hardware and budgets. Each notebook ran once on Kaggle with two NVIDIA T4 GPUs (about 15 GB each), four CPU cores and PyTorch 2.11 with CUDA 13.0, under a 10.8-hour ceiling within the 12-hour session. TSR trained the teacher for 170 minutes, the `low` 3x student for 190 and the `igpu` 3x and `low` 2x students in parallel for 110 (10.13 hours in total). FG trained `high` for 150 minutes, `low` for 200 and `igpu` and `mid` in parallel for 120 (10.07 hours). Ablations ran for 30 minutes each, two at a time. Checkpoints were chosen by validation PSNR averaged over inputs with and without engine data.

Test suites and modes. TSR is tested at 3x and 2x on synthetic scenes from held-out textures, held-out TartanAir environments, held-out Sintel scene families, Vid4 and 1080p Vimeo validation videos, plus native CARLA renders at 2x. Every suite runs in two modes. In engine mode the input is a jittered single-sample render with exact motion vectors where they exist (synthetic, TartanAir, Sintel) or RAFT motion otherwise. In plain mode the input is a bicubic downscale with no jitter, depth or motion vectors. FG is tested on Vimeo-90K triplets, TartanAir, Sintel, Vid4, Vimeo-1080p clips at 544 lines and at native 1072 lines, synthetic suites for UI, menus, cuts and camera reversal, and, after the run, the 12 Middlebury sequences with ground-truth middle frames [40]. FG modes are interpolation, extrapolation ($\tau = 1.5$) and 3x multi-frame generation, each with and without engine hints.

Baselines and metrics. TSR baselines are jitter-aware bilinear and bicubic upsampling and an analytic TAA upscaler (history reprojection, a 3×3 YCoCg neighbourhood clamp and a 10 percent blend). FG baselines are frame repetition, frame averaging, motion-vector reprojection and RIFE 4.26 unchanged. We report PSNR and SSIM [41] on luma (PSNR-Y) for TSR and PSNR on RGB for FG, LPIPS, FLIP [42] for static convergence, a temporal-change error (the loss above as a metric), tOF [23], a UI ghosting rate (share of UI pixels off by more than 10 percent) and the gated share of the cut gate. Latency is the mean of timed fused FP16 forward passes on one T4. A pacing model simulates a GPU-bound loop with just-in-time CPU scheduling on a variable-refresh display and reports displayed frame rate, game latency (input to a real frame that reflects it) and camera latency (input to any frame whose camera reflects it).

Gated samples. The test clips contain no cuts, so every real sample that the gate replaces by a repeated frame is a false alarm. FG tables

Table 2 TSR at 3x, PSNR-Y (dB) per suite. Plain mode: bicubic downscale, no jitter or engine data. Engine mode: jittered render with motion vectors (exact for synthetic, TartanAir and Sintel, RAFT for video) and depth where available. Bold marks the best deployable tier per row (teacher excluded)

	Plain video				Engine data			
Suite	Bicubic	`igpu`	`low`	Teacher	TAA	`igpu`	`low`	Teacher
Synthetic game	31.23	32.65	**33.42**	34.17	29.35	32.72	**34.27**	35.55
TartanAir	32.34	33.55	**34.03**	34.34	30.64	33.00	**33.43**	33.81
Sintel	33.38	34.23	**34.64**	34.99	31.07	33.70	**34.48**	35.08
Vid4	25.24	26.52	**27.00**	27.48	23.96	26.75	**27.32**	28.20
Vimeo-1080p	35.77	36.50	**36.96**	37.33	33.43	36.88	**37.77**	38.57

Table 3 TSR ablations at 3x: PSNR-Y difference (dB) to a reference trained with the same 30-minute budget. P plain mode, E engine mode

Removed	Synthetic	TartanAir	Vid4
History, E	−1.94	−0.71	−2.04
G-buffers, E	−1.75	−0.72	−1.06
Video training, P	−1.58	−1.18	−1.26
Video training, E	+0.45	+0.17	+0.29
Motion net, P	−0.16	−0.21	−0.46
16-frame unrolls, P	−0.56	−0.23	−0.44
16-frame unrolls, E	−0.18	−0.02	−0.08

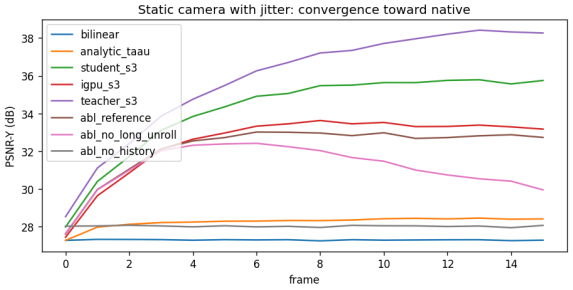


Fig. 2 Static camera with sub-pixel jitter on the synthetic suite: PSNR-Y against frame index for the baselines, the trained tiers and three 30-minute ablations (`abl_reference` is the ablation baseline)

therefore report samples the gate left alone ("not gated"), and the gate is evaluated separately.

Use of AI tools. A large language model assistant (Anthropic Claude) was used to help write the code, analyse the result files and draft this manuscript. Every number in the paper is taken from the result files of the runs, which are public with the code.

7 Results

7.1 Temporal super-resolution

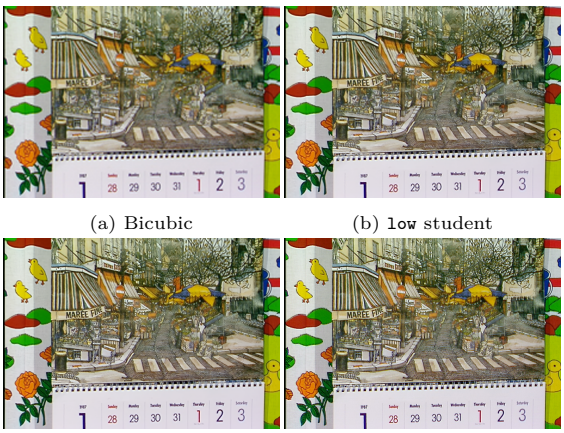
Table 2 gives the 3x results. On plain video the `low` student beats bicubic upscaling on every suite by 1.19 to 2.19 dB PSNR-Y, lowers LPIPS from 0.205 to 0.400 to 0.075 to 0.147, and wins 27 of 28 real clips (the exception is a Vimeo clip where bicubic already reaches 52.8 dB). With engine data it beats the analytic TAA upscaler by 2.79 to 4.92 dB, on all 28 real clips, and with lower temporal-change error on all five suites (for example 0.0229 against 0.0236 on Vid4 and 0.0109 against 0.0110 on synthetic scenes). In plain mode its temporal-change error is lower than bicubic's on three suites and higher on Sintel (0.0090 against 0.0081) and Vimeo-1080p (0.0060 against 0.0059). The teacher, at ten times the cost, adds 0.4 to 1.3 dB in engine mode and 0.3 to 0.8 dB in plain mode. The 6.2-GMAC `igpu` student keeps most of the gain. At 2x the student leads bicubic by 1.6 to 2.7 dB in plain mode, and on native CARLA renders it scores 32.60 dB against 32.33 for bicubic and 30.83 for TAA.

Static convergence. With a static camera and sub-pixel jitter a temporal upscaler should approach the native image. After 16 frames the student reaches 35.75 dB (FLIP 0.025), the teacher 38.26 dB (0.020), TAA 28.42 dB and the single-frame ablation 28.08 dB (Fig. 2). Without the 16-frame unrolls the model peaks near frame 6 and falls to 29.96 dB. In plain mode there is no jitter to accumulate and the student leads bicubic on static scenes by only 0.73 dB.

Ablations. Table 3 isolates the components with 30-minute runs of the `low` recipe. History and G-buffers are the largest engine-mode effects (0.7 to 2.0 dB). Plain-video training is worth 1.2

Table 4 FG interpolation on plain video (no engine hints), PSNR (dB). Video suites use samples the cut gate left alone (count in brackets). Middlebury was measured after the run on CPU with the downloaded weights

Suite	Average	RIFE 4.26	`igpu`	`low`	`mid`	`high`
Middlebury (12)	28.02	33.34	30.30	31.56	33.83	**34.23**
Vimeo-90K triplet (45)	26.36	34.87	29.69	32.94	34.38	**35.33**
Vimeo-1080p, 544 lines (9)	32.47	36.93	32.89	34.93	36.52	**37.14**
Vimeo-1080p, native 1072 lines (5)	29.37	33.00	29.58	31.21	33.01	**33.24**
Vid4 (4)	24.09	28.21	26.23	26.72	28.66	**29.05**
TartanAir (9)	21.29	**30.18**	22.08	25.23	28.64	30.11
Sintel (8)	23.60	**26.88**	23.75	24.48	25.97	26.38



(a) Bicubic (b) `low` student

(c) Teacher (d) Ground truth

Fig. 3 Vid4 calendar at 3x in plain mode (bicubic downscale, no motion vectors). The student sharpens the sign lettering, awning stripes and calendar digits that bicubic upscaling blurs, but adds a fine grid texture on the cobblestones, and the teacher recovers more detail

Table 5 FG extrapolation ($\tau = 1.5$), PSNR (dB), samples the cut gate left alone. Plain: no engine hints. Engine: with motion vectors, depth and camera prior, against motion-vector reprojection (MV)

Plain	Repeat	RIFE	`low`	`high`
Vimeo-1080p	30.15	27.27	31.31	**33.38**
Native 1080p	28.79	27.01	28.94	**30.65**
Vid4	21.41	21.95	23.93	**25.67**
TartanAir	19.37	19.51	22.28	**24.94**
Sintel	21.90	21.97	22.94	**24.02**
Engine	**MV**		**`low`**	**`high`**
Vimeo-1080p	32.03		32.11	**32.82**
Native 1080p	29.86		29.86	**30.63**
Vid4	25.89		24.99	**25.92**
TartanAir	**26.91**		24.77	25.12
Sintel	**24.20**		24.08	24.14
Synthetic	32.88		33.56	**34.38**

to 1.6 dB on plain video and costs nothing measurable on render input (the model without it is 0.2 to 0.5 dB better there). The motion estimator adds 0.2 to 0.5 dB on plain video. A repeat of the whole configuration a day earlier with shorter budgets (teacher 140, student 120 minutes) gave test means within 0.3 dB of these, so effects below a few tenths of a dB should be read as noise. Figure 3 shows Vid4 in plain mode.

7.2 Frame generation

Interpolation. Table 4 compares the tiers with RIFE 4.26 on plain video. The fine-tuned `high` tier beats RIFE on Middlebury (34.23 against 33.34 dB) and on four of the six video suites by 0.21 to 0.84 dB, ties on TartanAir and loses 0.50 dB on the eight Sintel samples. The `mid` tier (44.3 GMAC) is within 0.5 dB of RIFE on five of the seven suites. The `low` tier beats frame averaging everywhere but trails RIFE by 1.5 to 5.0 dB, and `igpu` is barely above averaging on the 1080p suites. With engine hints `high` is within −0.6 to +0.7 dB of RIFE without them, so motion vectors add little to interpolation. Motion-vector reprojection is 0.6 to 3.0 dB below RIFE on the real suites. In 3x multi-frame generation `high` matches or beats RIFE on four of five suites (Vimeo-1080p 37.07 against 37.01 dB, native 1080p 35.20 against 34.76).

Extrapolation. Table 5 shows the mode that adds no held frame. RIFE is not built for $\tau > 1$ and stays within 0.6 dB of frame repetition or below it. On plain video `high` beats repetition by 1.9 to 5.6 dB and `low` by 0.2 to 2.9 dB. With engine motion vectors plain reprojection remains better on TartanAir (1.8 dB, exact motion) and ties on Sintel and Vid4.



(a) Target (b) Average (c) RIFE 4.26 (d) `low` (e) `low` + UI layer (f) Predicted UI mask

Fig. 4 Interpolation of a synthetic scene with HUD bars, a crosshair and a changing subtitle. Averaging and RIFE blend the old and new subtitle text, the `low` tier copies the UI region from the newest real frame using its predicted mask (f)

UI, camera reversal and cuts. On synthetic HUD and subtitle scenes (interpolation with engine hints) the UI ghosting rate is 0.047 for `low` and 0.049 for `high`, against 0.228 for RIFE and 0.302 for reprojection, and 0.010 when the engine supplies a UI layer (Fig. 4). Removing UI training raises it to 0.362. On rapid camera reversals in extrapolation the late-latched camera prior raises `low` from 25.30 to 28.01 dB and `high` from 26.30 to 27.07 dB, while repetition scores 25.91 dB. The gate detected all 48 synthetic cuts in both input modes. With motion vectors it false-alarmed on 0 to 8 percent of real samples (20 percent of the ten Sintel samples). Without them it false-alarmed on 0 to 25 percent of two-frame real samples, on 30 to 50 percent of 3x multi-frame samples, whose real frames are three apart, and on 9 percent of continuous synthetic pairs against its 1 percent calibration target. A threshold calibrated on adjacent synthetic pairs does not transfer to real content or wider gaps.

Ablations. Thirty-minute runs of the `low` recipe show that the RIFE weights matter most on plain video: without inheritance Vimeo-90K interpolation drops from 28.01 to 25.47 dB. Without engine hints in training, extrapolation with exact motion on TartanAir falls from 24.24 to 19.04 dB, but plain-video interpolation rises from 28.01 to 29.19 dB, so the hints cost plain-video quality at this budget. Distillation adds 0.2 to 1.3 dB. A model trained only for extrapolation fails at interpolation (21.29 against 32.73 dB with engine hints).

7.3 Real-time cost

Table 1 gives the measured T4 times. They are far above the arithmetic cost: the 12.7-GMAC FG `low` tier takes 48.1 ms and the 6.2-GMAC TSR `igpu` tier 16.3 ms, and cuDNN autotuning changed neither compared with the previous run (47.5 and 17.0 ms). The `mid` and `high` FG tiers are faster at 1440p (85.7 and 91.2 ms) than at 1080p (106.6 and 120.4 ms) because their flow is halved above 1152 lines, so full-resolution flow, not the head, dominates their time. With 48.1 ms per generated frame the pacing model at a 30 fps render rate drops the displayed rate to 24.6 fps in every FG mode. Interpolation raises game latency from 38.3 to 127.1 ms, while extrapolation keeps it at 38.3 ms and late latching brings camera latency to 43.7 ms. The fused networks export to ONNX with a maximum ONNX Runtime difference of 2.8×10^{-5} (TSR) and 7.5×10^{-6} (FG).

8 Discussion

What one model per task achieves. A single TSR network handles jittered renders with engine data and plain video without it. Training on both degradations is what makes plain video work, and it costs the render path nothing measurable (Table 3). Longer budgets no longer help: the student trained 58 percent longer than in the previous run and moved by at most 0.3 dB, so the next TSR gains need more exact-motion data rather than more time. For FG, fine-tuning an open interpolation network with zero-initialised hint channels keeps its quality and adds what video interpolation lacks for games: extrapolation, UI handling and a camera prior.

Where it falls short. The tiers that fit integrated and entry GPUs do not reach RIFE 4.26's quality in FG. Moving their flow to full resolution in training raised `low` by 3.4 dB on Middlebury, yet it stays 1.8 dB below RIFE at half of RIFE's channel widths and a sixth of its cost, so capacity now limits it. Extrapolation with exact motion vectors does not beat plain reprojection on TartanAir. The histogram cut gate is not deployable.

Most importantly for real-time use, every measured tier exceeds a 16.7 ms frame budget in eager PyTorch on a T4 except the TSR `igpu` tier, which takes 16.3 ms (10.0 ms with engine motion vectors), nearly the whole budget. The TSR `low` tier's 27.7 ms is 3.4 times its arithmetic estimate for a 4-TFLOPS integrated GPU, which points to per-kernel overhead and memory traffic. We infer this from the timings and have not profiled it.

Threats to validity. Each configuration ran once, and only the TSR configuration was repeated with other budgets. Several ungated FG means rest on few samples (four Vid4 and five native 1080p samples). Real training windows are about 91 percent Vimeo-90K and only 2 percent of TSR windows have exact motion. Low-resolution inputs are simulated from finished frames, so level-of-detail and shading-rate effects of real renders are missing, and lighting effects such as specular aliasing and reflections are absent from the synthetic data. The plain-video protocol uses clean bicubic downscaling, while real video adds codec artefacts. We did not compare with commercial upscalers or frame generators, which needs engine captures, and latency was measured on a data-centre GPU, not on target hardware.

9 Conclusion

NeuralSS shows that one compact recurrent network can upscale both engine renders and plain video, beating bicubic and analytic TAA upscaling on five suites, and that fine-tuning RIFE 4.26 with zero-initialised engine hints yields a frame generator that beats RIFE at its own width while adding extrapolation, UI masking and a camera prior. Everything was trained on free cloud GPUs from public data with audited splits. The results also mark what remains: frame-generation tiers small enough for integrated GPUs are 1.5 to 5 dB below RIFE, cut detection without motion vectors false-alarms, and in eager execution only the smallest super-resolution tier fits a 60 fps frame budget, with almost no margin. Next steps are wider half-resolution-flow FG tiers, cut thresholds calibrated on real clips per frame gap, fused compute-shader or TensorRT and DirectML runtimes timed on integrated GPUs, repeated seeds, and engine captures for a comparison with commercial systems.

Statements and Declarations

Funding. No funding was received for this work.

Competing interests. The author has no competing interests to declare.

Data availability. All datasets are public: DIV2K, Flickr2K, REDS, Vimeo-90K, Vid4 and MPI Sintel on Kaggle, and TartanAir, the CARLA-based GameIR renders, 1080p game frames and Vimeo-1080p on Hugging Face. The dataset identifiers, the split manifest, every metric file, plot and log of the runs reported here are in the project repository, https://github.com/parthmital/High-Throughput-Neural-Super-Sampling.

Code availability. The code, the three Kaggle notebooks with their executed outputs and the trained weights are in the same repository.

Ethics approval and consent. Not applicable.

Author contributions. Parth Mital designed the system, ran the experiments, analysed the results and wrote the manuscript.